SIMULATION EXPERIMENT – 3

➔ Objectives of this Experiment:

1. To understand the basic operation of I²C communication between an Arduino UNO and an LM75AD temperature sensor.
2. To develop an Embedded C program for acquiring temperature data from the LM75AD sensor.
3. To interface a pulse signal source with the digital input of the Arduino UNO.
4. To calculate pulse rate in beats per minute (BPM) from the acquired pulse signal.
5. To display the measured temperature and pulse rate through the Proteus Virtual Terminal.
6. To simulate and verify the health monitoring node using Proteus Design Suite.

➔ Expected Outcome from this Experiment:

Students shall be able to:

1. Configure the Arduino UNO for I²C-based sensor communication.
2. Interface the LM75AD temperature sensor using SDA and SCL lines.
3. Develop an Embedded C program for temperature acquisition using the Wire library.
4. Acquire and count digital pulse signals using an Arduino digital input.
5. Calculate pulse rate in BPM from the acquired pulse signal.
6. Display and verify temperature and pulse-rate information through serial communication.
7. Analyze the operation of a simple simulated health monitoring node.

➔ Software Required:

S. No.	Software	Purpose
1	Proteus Design Suite 8	Circuit design and simulation
2	Arduino IDE	Embedded C program development and HEX generation

➔ Hardware Components Used in Simulation:

S. No.	Component	Quantity
1	Arduino UNO	1
2	LM75AD Temperature Sensor	1
3	CLOCK – Animated Digital Squarewave Source	1
4	Resistor – 4.7 kΩ	2
5	Virtual Terminal	1
6	+5V Power Terminal	1

7	GND Power Terminal	As required

➔ **Objective Linkage:**

The experiment enables students to:

- Develop an Embedded C program for an Arduino UNO.
- Establish I²C communication between the Arduino UNO and LM75AD.
- Configure Arduino A4/SDA and A5/SCL for I²C communication.
- Acquire temperature data from an I²C temperature sensor.
- Acquire a digital pulse signal through Arduino digital pin D8.
- Calculate pulse rate in BPM.
- Transmit the acquired health parameters through the Arduino serial interface.
- Verify the complete health monitoring operation through Proteus simulation.

➔ **Theory:**

The **Arduino UNO** is based on the ATmega328P microcontroller and provides digital input/output pins and an I²C communication interface. I²C, or **Inter-Integrated Circuit**, is a two-wire serial communication protocol using **SDA (Serial Data)** and **SCL (Serial Clock)** lines.

In this experiment, an **LM75AD temperature sensor** is interfaced with the Arduino UNO using the I²C protocol. The Arduino UNO uses:

- **A4 / PC4 / SDA** for the SDA line.
- **A5 / PC5 / SCL** for the SCL line.

The LM75AD is configured with its address pins A0, A1 and A2 connected to ground. Therefore, the sensor operates at the I²C address:

0x48

Two **4.7 kΩ pull-up resistors** are connected between the +5V supply and the SDA/SCL lines because I²C communication requires the bus lines to be pulled HIGH when they are not being driven LOW.

The LM75AD temperature register is accessed by the Arduino using the standard Arduino Wire library. The Arduino requests two bytes of temperature data and processes the received value to obtain the temperature in degrees Celsius.

The second sensing input is a digital pulse signal generated using the Proteus **CLOCK** source. The CLOCK is configured to operate at **1.2 Hz**.

The corresponding pulse rate is:

BPM = 1.2 x 60 = 72 BPM

The pulse signal is connected to **Arduino D8**. The program detects the rising edges of the pulse signal and counts the number of pulses during a fixed measurement interval.

The measured temperature and calculated pulse rate are transmitted through the Arduino's serial TX pin and displayed on the **Proteus Virtual Terminal**.

The complete signal flow is therefore:

LM75AD Temperature Sensor

➔ **Experimental Procedure:**

A. Proteus Circuit Development

1. Open the Proteus Design Suite.
2. Create a new schematic design.
3. Select and place the **Arduino UNO**.
4. Select and place the **LM75AD** temperature sensor.
5. Connect **LM75AD pin 7 (SDA)** to Arduino **A4/SDA**.
6. Connect **LM75AD pin 2 (SCL)** to Arduino **A5/SCL**.
7. Connect LM75AD **A0 (pin 1)** to GND.
8. Connect LM75AD **A1 (pin 6)** to GND.
9. Connect LM75AD **A2 (pin 5)** to GND.
10. Leave the LM75AD **OS pin (pin 3)** unconnected.
11. Connect a **4.7 kΩ resistor** between +5V and the SDA line.
12. Connect another **4.7 kΩ resistor** between +5V and the SCL line.
13. Configure the LM75AD hidden power connections as:
    - GND → GND
    - VCC → +5V
14. Select and place the **CLOCK** component.
15. Set the CLOCK frequency to **1.2 Hz**.
16. Connect the CLOCK output to Arduino **D8**.
17. Select and place the **Virtual Terminal**.
18. Connect Arduino **D1/TX** to the Virtual Terminal **RXD**.
19. Configure the Virtual Terminal for **9600 baud**.
20. Leave the Virtual Terminal TXD, RTS and CTS unconnected.
21. Complete and verify all circuit connections.
22. Develop the Arduino Embedded C program.

23. Compile the program using the Arduino IDE.
24. Generate the corresponding HEX file.
25. Load the HEX file into the Arduino UNO component in Proteus.

B. Program Execution and Verification

1. Start the Proteus simulation.
2. Observe the LM75AD temperature value.
3. Verify that the Arduino communicates with the LM75AD through I^2C.
4. Observe the digital pulse signal generated by the CLOCK source.
5. Verify that the pulse signal is received at Arduino D8.
6. Observe the temperature information on the Virtual Terminal.
7. Observe the calculated pulse rate in BPM.
8. Verify that the pulse rate is approximately **72 BPM** for a 1.2 Hz pulse source.
9. Change the LM75AD temperature parameter during simulation.
10. Observe the corresponding change in the displayed temperature.
11. Verify continuous temperature acquisition and pulse-rate calculation.
12. Record the simulated output values.

➔ **PROGRAM:**

```
#include <Wire.h>

#define LM75_ADDRESS 0x48
#define PULSE_PIN 8

unsigned long pulseCount = 0;
unsigned long lastTempTime = 0;
unsigned long lastBpmTime = 0;

int previousPulseState = LOW;

float readTemperature()
{
  Wire.beginTransmission(LM75_ADDRESS);
```

```
  Wire.write(0x00);
  Wire.endTransmission();

  Wire.requestFrom(LM75_ADDRESS, 2);

  if (Wire.available() >= 2)
  {
    byte msb = Wire.read();
    byte lsb = Wire.read();

    int16_t rawTemperature =
      ((int16_t)msb << 8) | lsb;

    return rawTemperature / 256.0;
  }

  return -999.0;
}

void setup()
{
  Serial.begin(9600);
  Wire.begin();

  pinMode(PULSE_PIN, INPUT);

  Serial.println("I2C HEALTH MONITORING NODE");
  Serial.println("--------------------------");
}

void loop()
```

```
{
  unsigned long currentTime = millis();

  /* Pulse counting */

  int currentPulseState = digitalRead(PULSE_PIN);

  if (currentPulseState == HIGH &&
     previousPulseState == LOW)
  {
   pulseCount++;
  }

  previousPulseState = currentPulseState;

  /* Temperature acquisition every 1 second */

  if (currentTime - lastTempTime >= 1000)
  {
   lastTempTime = currentTime;

   float temperature = readTemperature();

   Serial.print("Temperature: ");

   if (temperature > -100)
   {
    Serial.print(temperature, 1);
    Serial.println(" C");
   }
   else
```

```
  {
    Serial.println("Sensor Error");
  }
 }

 /* BPM calculation every 5 seconds */

 if (currentTime - lastBpmTime >= 5000)
 {
  lastBpmTime = currentTime;

  float bpm = (pulseCount * 60.0) / 5.0;

  Serial.print("Pulse Rate: ");
  Serial.print(bpm, 0);
  Serial.println(" BPM");

  Serial.println("---------------------------");

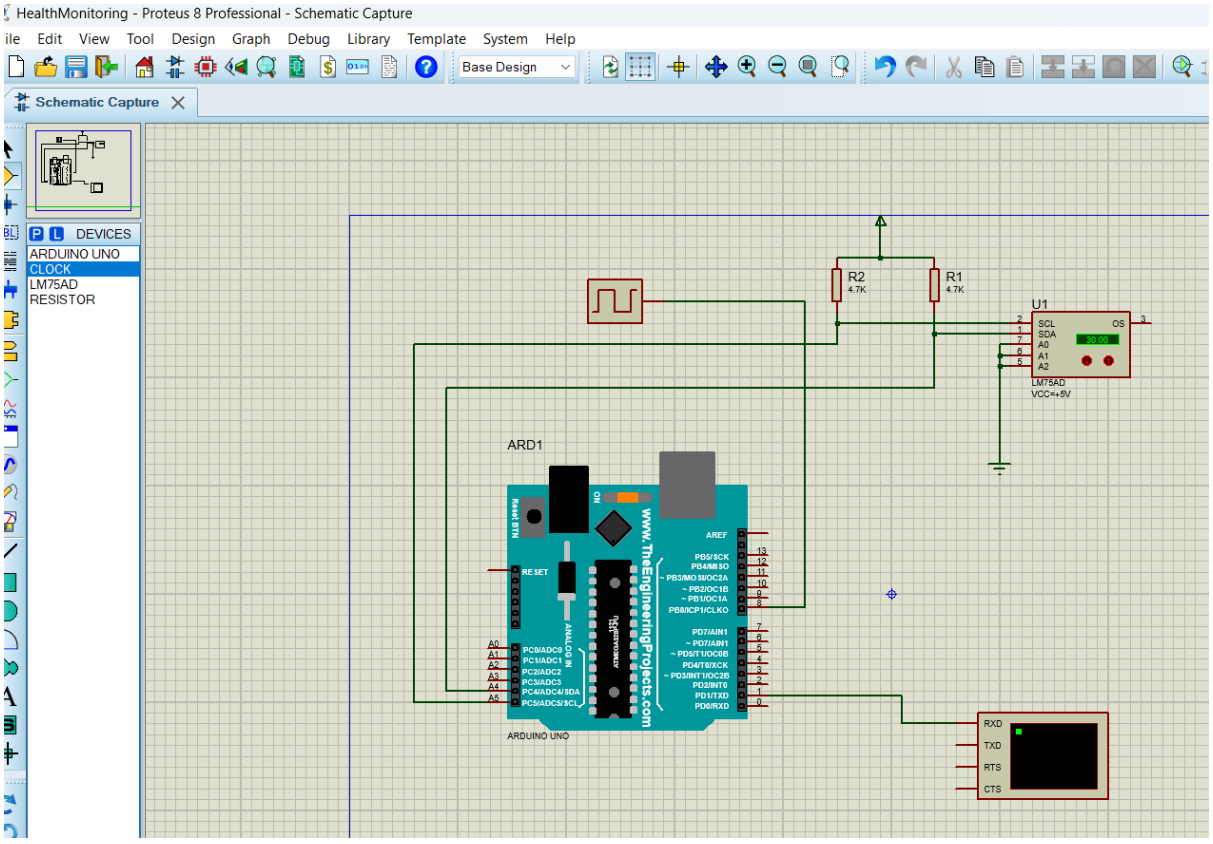
  pulseCount = 0;
 }
}
```

➔ **Program Explanation:**

Instruction / Section	**Function**
#include <Wire.h>	Includes the Arduino I²C communication library
LM75_ADDRESS 0x48	Defines the I²C address of the LM75AD
PULSE_PIN 8	Defines Arduino D8 as the pulse input
Wire.begin()	Initializes I²C communication
Serial.begin(9600)	Initializes serial communication at 9600 baud
pinMode(PULSE_PIN, INPUT)	Configures D8 as a digital input

Wire.beginTransmission()	Starts communication with the LM75AD
Wire.write(0x00)	Selects the LM75AD temperature register
Wire.requestFrom()	Requests temperature data from the sensor
Wire.read()	Reads the received temperature bytes
digitalRead(PULSE_PIN)	Reads the pulse input
Rising-edge condition	Detects a new pulse
pulseCount++	Increments the pulse counter
millis()	Provides timing for periodic processing
bpm = (pulseCount × 60)/5	Calculates pulse rate in BPM
Serial.print()	Sends the measured values to the Virtual Terminal
pulseCount = 0	Resets the pulse count for the next measurement interval

➔ CIRCUIT DIAGRAM:



Arduino UNO – LM75AD I²C Health Monitoring Node

Connection:

Arduino UNO	LM75AD / Component
A4 / SDA	LM75AD Pin 7 – SDA

A5 / SCL	LM75AD Pin 2 – SCL
GND	LM75AD Pin 1 – A0
GND	LM75AD Pin 6 – A1
GND	LM75AD Pin 5 – A2
D8	CLOCK output
D1 / TX	Virtual Terminal RXD
+5V	4.7 kΩ pull-up resistor to SDA
+5V	4.7 kΩ pull-up resistor to SCL

Pulse and Serial Connections:

CLOCK 1.2 Hz ——————▶ Arduino D8

Arduino D1/TX ——————▶ Virtual Terminal RXD

➔ INPUT PARAMETER TABLE – VALUES TO BE ENTERED

S. No.	Input Parameter	Symbol / Setting	Value to Enter	Unit
1	Microcontroller	Arduino UNO	Arduino UNO	—
2	Temperature Sensor	LM75AD	LM75AD	—
3	I²C Address	Address	0x48	—
4	SDA Pin	Arduino	A4/SDA	—
5	SCL Pin	Arduino	A5/SCL	—
6	SDA Pull-up	R1	4.7	kΩ
7	SCL Pull-up	R2	4.7	kΩ
8	Pulse Input	Digital Pin	D8	—
9	CLOCK Frequency	Pulse Source	1.2	Hz
10	Expected Pulse Rate	BPM	72	BPM
11	Serial Baud Rate	Virtual Terminal	9600	baud
12	Temperature Initial Value	LM75AD	25.00	°C
13	BPM Measurement Interval	Program	5	seconds
14	Temperature Sampling Interval	Program	1	second

➔ OUTPUT PARAMETER TABLE – VALUES TO BE RECORDED

S. No.	Output Parameter	Expected Value / Condition	Practical / Simulated Observation	Status
1	I²C Device Address	0x48 detected		Verified / Not Verified
2	Temperature Acquisition	Temperature displayed in °C		Verified / Not Verified
3	Temperature Variation	Display follows LM75AD setting		Verified / Not Verified
4	Pulse Input	Digital pulses received at D8		Verified / Not Verified
5	Pulse Rate	Approximately 72 BPM		Verified / Not Verified
6	Serial Communication	Data displayed on Virtual Terminal		Verified / Not Verified
7	Continuous Monitoring	Temperature and BPM repeatedly displayed		Verified / Not Verified

➔OUTPUT:

